

Delhi Daily Temperature Forecasting Using Foundation Time Series Models

Data Preprocessing · Feature Engineering · Model Evaluation

Technical Report

Generated: May 14, 2026

Dataset: Kaggle — Daily Delhi Climate Dataset

Temporal Coverage: January 1, 2013 — April 24, 2017

Location: New Delhi, India

1. Dataset Description

The dataset is the **Daily Delhi Climate** dataset sourced from Kaggle. It records four meteorological variables measured daily in New Delhi, India, spanning **January 1, 2013 to April 24, 2017** — a total of approximately **1,462 daily observations** across 5 columns.

Column	Data Type	Unit	Role	Description
date	DateTime	—	Index	Calendar date (YYYY-MM-DD). Used as time index.
meantemp	Float	°C	TARGET	Mean daily temperature. Primary variable to forecast.
humidity	Float	%	Feature	Mean daily relative humidity.
wind_speed	Float	km/h	Feature	Mean daily wind speed.
meanpressure	Float	hPa	Feature	Mean daily atmospheric pressure.

Table 1 — Dataset columns, units, and roles.

New Delhi experiences a strong semi-arid climate with a marked annual temperature cycle. Winter temperatures can drop to approximately 8–10 °C (December–January), while peak summer temperatures exceed 38–40 °C (May–June). This wide seasonal swing creates a challenging yet well-structured time-series forecasting scenario with clear annual periodicity.

1.3 Dataset Statistics

Attribute	Value
Total rows	~1,462 daily observations
Total columns	5 (1 index + 4 features)
Time range	2013-01-01 → 2017-04-24
Temporal resolution	Daily (enforced via asfreq('D'))
Target variable	meantemp (°C)
Approximate temp range	8 °C (winter) — 40 °C (summer)
Source	Kaggle — Daily Delhi Climate Dataset

Table 2 — High-level dataset statistics.

2. Preprocessing Pipeline

Preprocessing was applied in strict chronological order to prevent data leakage. All statistics (IQR bounds, etc.) are derived **exclusively from the training split** and then applied to both splits.

Step 1 — Data Loading & Frequency Enforcement

The CSV file is read with `pd.read_csv()`. The **date** column is parsed with `pd.to_datetime()` and set as the DataFrame index. `.asfreq('D')` is then applied to enforce a daily frequency, inserting NaN for any missing calendar dates to guarantee a contiguous time series.

Step 2 — STL Decomposition (Exploratory)

Seasonal-Trend decomposition using LOESS (STL) is applied to the full series (`period=365`, `robust=True`) to visualise and understand the underlying structure before any split. The decomposition separates the series into three additive components: **Trend** (long-term direction), **Seasonality** (365-day annual pattern), and **Residual** (noise). This step is exploratory only — no transformation is applied to the data.

Step 3 — Augmented Dickey–Fuller (ADF) Stationarity Test

The ADF test (`autolag='AIC'`) checks the null hypothesis H_0 : the series contains a unit root (non-stationary). Decision rule: $p\text{-value} < 0.05 \rightarrow \text{reject } H_0 \rightarrow \text{stationary}$. If the raw series is non-stationary, **first-order differencing** (subtracting each value from the previous) is applied and the test is repeated. Output includes the ADF statistic, p-value, number of lags used, and critical values at 1%, 5%, and 10%.

Step 4 — Chronological Train / Test Split (80 / 20)

The dataset is split into training (first 80 % $\approx 1,170$ rows) and test (last 20 % ≈ 292 rows) sets using index-based slicing (`.iloc[:split_idx] / .iloc[split_idx:]`). **No shuffling** is performed — temporal order is strictly preserved. `.copy()` is called on both slices to prevent silent aliasing. This split is performed **before** any statistics are computed, ensuring zero data leakage.

Step 5 — Outlier Detection & Imputation

Two strategies are used depending on the feature:

5a — meanpressure: Domain-Knowledge Threshold

Values outside the physically possible atmospheric range (< 900 hPa or $> 1,100$ hPa) are flagged as outliers. This threshold is based on meteorological domain knowledge and is therefore safe to apply before the split without leakage.

5b — meantemp, humidity, wind_speed: IQR Method

Inter-Quartile Range bounds are calculated **from the training set only**:

Formula	Description
<code>Q1 = train[col].quantile(0.25)</code>	25th percentile of training values
<code>Q3 = train[col].quantile(0.75)</code>	75th percentile of training values

Formula	Description
$IQR = Q3 - Q1$	Spread of the middle 50 %
$Lower = Q1 - 1.5 \times IQR$	Lower fence — anything below is an outlier
$Upper = Q3 + 1.5 \times IQR$	Upper fence — anything above is an outlier

Table 3 — IQR outlier detection formulas.

Detected outliers in both train and test sets are replaced with NaN. Gaps are then filled using a three-step imputation chain: `.interpolate()` (linear interpolation between neighbours) → `.ffill()` (forward fill for trailing edges) → `.bfill()` (backward fill for leading edges). The result is a clean, gap-free time series in both splits.

3. Feature Engineering

Feature engineering expands the original 4 columns into 15 informative predictors by adding temporal, autoregressive, and statistical context. All features are computed **after** the train/test split and outlier handling. A dedicated function `add_features(df_split, train_tail=None)` handles both splits to avoid leakage.

Leakage prevention: The test set is seeded with the last 30 rows of the training set (`train_tail=train.iloc[-30:]`). These seed rows are dropped after feature computation, so only true test-period rows remain with fully valid rolling/lag features.

Summary: 4 original features + 11 engineered features = **15 total features**.

3.1 Temporal / Cyclical Features (4 features)

These features encode the position of each observation within the calendar year. Cyclical (sin/cos) encoding is preferred over raw integers because it preserves the circular nature of time: day 365 and day 1 are adjacent, not 364 apart.

Feature	Formula	Range	Rationale
month	<code>df.index.month</code>	1 – 12	Coarse monthly label; captures month-level seasonality (e.g., April vs. December).
day_of_year	<code>df.index.day_of_year</code>	1 – 365	Granular yearly position; finer resolution than month alone.
sin_doy	$\sin(2\pi \times \text{day_of_year} / 365)$	[-1, 1]	Projects annual cycle onto unit circle. Ensures day 365 and day 1 are numerically close (no discontinuity at year boundary).
cos_doy	$\cos(2\pi \times \text{day_of_year} / 365)$	[-1, 1]	Complement of <code>sin_doy</code> . Together, sin and cos encode the full 360° position on the annual cycle.

Table 4 — Temporal / cyclical features.

Why sin/cos instead of raw day-of-year? A linear day-of-year value assigns a numeric distance of 364 between day 1 (January 1) and day 365 (December 31), even though they are consecutive days. Sine/cosine encoding maps the year onto a circle where this distance is approximately 0, correctly representing seasonal continuity across the year boundary.

3.2 Lag / Autoregressive Features (3 features)

Lag features provide the model with direct access to past temperature values, capturing autocorrelation in the time series. `shift(n)` shifts the series forward by n steps so that the value at row t equals the original value at row $t - n$.

Feature	Formula	Lag	Rationale
temp_lag_1	<code>meantemp.shift(1)</code>	1 day	Yesterday's temperature is typically the strongest single predictor of today's temperature due to high day-to-day autocorrelation.
temp_lag_7	<code>meantemp.shift(7)</code>	7 days	One-week lag captures weekly weather cycles and atmospheric patterns that tend to repeat on a ~7-day timescale.
temp_lag_14	<code>meantemp.shift(14)</code>	14 days	Two-week lag captures slower synoptic-scale patterns and medium-term seasonal transitions.

Table 5 — Lag / autoregressive features.

3.3 Rolling Statistics (4 features)

Rolling features summarise temperature behaviour over a sliding window of past observations. All windows are computed on `meantemp.shift(1)` — the shifted series — so today's value is never included in its own window, preventing look-ahead bias.

Feature	Formula	Window	Rationale
temp_rollmean_7	<code>shift(1).rolling(7).mean()</code>	7 days	Short-term smoothed temperature trend. Reduces noise and highlights the direction of change over the past week.
temp_rollstd_7	<code>shift(1).rolling(7).std()</code>	7 days	Week-scale temperature volatility. A high value indicates rapidly changing weather conditions.
temp_rollmean_30	<code>shift(1).rolling(30).mean()</code>	30 days	Monthly smoothed trend. Captures seasonal momentum and gradual climate transitions (e.g., winter → spring warming).

Feature	Formula	Window	Rationale
temp_rollstd_30	shift(1).rolling(30).std()	30 days	Month-scale volatility. Elevated values indicate a period of seasonal transition with high day-to-day variability.

Table 6 — Rolling statistic features.

Note on shift-before-roll: Without `.shift(1)`, the rolling window at time t would include the temperature value at t itself. This would create a circular dependency where the target leaks into its own feature. Applying `shift` first guarantees that the window at time t covers only observations $t-1$, $t-2$, ..., $t-w$.

3.4 Feature Engineering Summary

Group	Count	Feature Names
Original (raw)	4	meantemp, humidity, wind_speed, meanpressure
Temporal / Cyclical	4	month, day_of_year, sin_doy, cos_doy
Lag / Autoregressive	3	temp_lag_1, temp_lag_7, temp_lag_14
Rolling Statistics	4	temp_rollmean_7, temp_rollstd_7, temp_rollmean_30, temp_rollstd_30
TOTAL	15	—

Table 7 — Complete feature inventory after engineering.

4. Train / Test Split Strategy

Time-series data must never be split randomly because doing so would allow the model to train on future information. A strict **chronological 80 / 20 split** is used.

Split	Ratio	Approx. Rows	Approximate Date Range
Training set	80 %	~1,170 days	January 1, 2013 — December 2015
Test set	20 %	~292 days	January 2016 — April 24, 2017

Table 8 — Train / test split sizes.

4.1 Walk-Forward Rolling Forecast

Rather than making a single static 292-day prediction, the models use a **walk-forward (rolling) forecasting strategy** to simulate real-world deployment conditions:

Step	Action
1	Use the full training context (up to the model's context cap) to predict the next 14 days.
2	Append the 14 actual (ground-truth) observations to the context.

3	Repeat until all 292 test days are covered (~21 iterations of 14 days).
4	Concatenate all 14-day chunks into the final 292-day forecast array.

Table 9 — Walk-forward rolling forecast procedure.

Why walk-forward? Updating the context with actual observed values after each prediction window prevents compounding errors that occur when a model's own (potentially imperfect) predictions are fed back as future context. It closely mirrors how a deployed forecasting system would operate.

4.2 Model-Specific Context Caps

Model	Context Cap	Implication
Chronos-T5-Tiny	512 days	Uses most recent 512 days of history at each step.
Chronos-T5-Small	2,048 days	Can see the entire training set at once (1,170 days < 2,048).
Chronos-Bolt-Small	512 days	Uses most recent 512 days of history at each step.

Table 10 — Context window caps per model.

5. Models

All three models used in this project belong to the **Chronos** family, developed by Amazon Research. They are large neural networks pre-trained on millions of diverse real-world time series. They operate in a **zero-shot** manner — no fine-tuning on the Delhi dataset is performed. The models receive a normalised context window and return a probabilistic (or deterministic) forecast.

5.1 Input Normalisation (All Models)

All Chronos models were trained on z-score normalised data. Normalisation is applied before every prediction call and reversed afterwards:

Step	Formula	Purpose
Forward	$(x - \mu) / (\sigma + \epsilon)$	Scale context to zero-mean, unit-variance. $\epsilon = 1 \times 10^{-8}$ prevents division by zero.
Inverse	$\text{prediction} \times (\sigma + \epsilon) + \mu$	Convert model output back to original °C scale.

Table 11 — Normalisation applied before/after each prediction.

5.2 Chronos-T5-Tiny

HuggingFace identifier: amazon/chronos-t5-tiny

Attribute	Value
Parameters	~8 million
Architecture	T5 encoder-decoder transformer (adapted for time series)
Context window cap	512 days

Prediction type	Probabilistic — generates 20 independent sample paths
Final prediction	Median of 20 samples (denormalised)
API class	ChronosPipeline
Inference speed	Fastest (lowest memory footprint)

Best suited for rapid iteration and environments with limited GPU memory. With only ~8 M parameters it sacrifices some capacity for speed.

5.3 Chronos-T5-Small

HuggingFace identifier: `amazon/chronos-t5-small`

Attribute	Value
Parameters	~46 million
Architecture	T5 encoder-decoder transformer
Context window cap	2,048 days — can ingest the entire training set
Prediction type	Probabilistic — 20 sample paths
Final prediction	Median of 20 samples (denormalised)
API class	ChronosPipeline
Inference speed	Slowest (highest GPU memory usage)

Highest modelling capacity among the three. Its 2,048-day context window allows it to see the complete ~1,170-day training history in a single pass, giving it the longest temporal memory.

5.4 Chronos-Bolt-Small

HuggingFace identifier: `amazon/chronos-bolt-small`

Attribute	Value
Parameters	~46 million
Architecture	Optimised / distilled Chronos variant
Context window cap	512 days
Prediction type	Deterministic — single forecast path (no num_samples)
Final prediction	Direct output (no aggregation required)
API class	BaseChronosPipeline
Inference speed	Fast (optimised inference engine)

Production-oriented variant. Bolt uses a distilled inference engine that trades probabilistic uncertainty estimates for faster, deterministic predictions. Suitable for latency-sensitive applications.

5.5 Model Comparison at a Glance

Model	Params	Context Cap	Output Type	Speed
Chronos-T5-Tiny	~8 M	512 days	Probabilistic (median of 20)	Fastest
Chronos-T5-Small	~46 M	2,048 days	Probabilistic (median of 20)	Slowest
Chronos-Bolt-Small	~46 M	512 days	Deterministic (single path)	Fast

Table 15 — Side-by-side comparison of the three Chronos models.

6. Evaluation Metrics

Four complementary metrics are computed for each model on the 292-day test set. Together they cover magnitude of error, scale-free percentage error, and proportion of variance explained.

6.1 MSE — Mean Squared Error

Formula: $MSE = \text{mean}[(y - \hat{y})^2]$ | **Unit:** °C²

Primary loss for neural network training. Squaring errors means large deviations are penalised disproportionately. Less interpretable than RMSE due to squared units.

Range	$[0, \infty)$ — 0 is perfect
Sensitivity	High — dominated by large errors

6.2 RMSE — Root Mean Squared Error (Primary Ranking Metric)

Formula: $RMSE = \sqrt{MSE} = \sqrt{\text{mean}[(y - \hat{y})^2]}$ | **Unit:** °C

Square root of MSE restores the original unit (°C), making the error directly interpretable: 'on average, predictions deviate by X °C from actual temperature.' Models are ranked by RMSE ascending (lower = better).

Range	$[0, \infty)$ — 0 is perfect
Sensitivity	High — penalises large errors

6.3 sMAPE — Symmetric Mean Absolute Percentage Error (Secondary Metric)

Formula: $sMAPE = \text{mean}(|y - \hat{y}| / ((|y| + |\hat{y}|) / 2)) \times 100$ | **Unit:** %

Scale-free metric expressed as a percentage. The 'symmetric' formulation averages actual and predicted in the denominator, so over-prediction and under-prediction are penalised equally. Critical for this dataset because Delhi winter temperatures approach 0 °C — standard MAPE would produce extreme values and asymmetric penalties in those conditions.

Range	$[0, 200\%]$ — 0 % is perfect
Near-zero safe	Yes — denominator never collapses

6.4 R² — Coefficient of Determination

Formula: $R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$ where $SS_{\text{res}} = \sum (y - \hat{y})^2$, $SS_{\text{tot}} = \sum (y - \bar{y})^2$ | **Unit:** — (dimensionless)

Measures the proportion of variance in the actual temperatures that is explained by the model's predictions. $R^2 = 1.0 \rightarrow$ perfect forecast. $R^2 = 0.0 \rightarrow$ no better than always predicting the mean. $R^2 < 0.0 \rightarrow$ worse than predicting the mean.

Range	$(-\infty, 1]$ — 1 is perfect
Baseline	$R^2 = 0$ corresponds to the naive mean predictor

6.5 Metrics Summary

Metric	Formula (compact)	Unit	Best Value	Role
MSE	$\text{mean}((y - \hat{y})^2)$	°C ²	0	Loss function; penalises large errors
RMSE	$\sqrt{\text{MSE}}$	°C	0	Primary ranking; interpretable scale
sMAPE	$ y - \hat{y} / \text{avg}(y , \hat{y}) \times 100$	%	0 %	Scale-free; robust at near-zero values
R ²	$1 - SS_{\text{res}} / SS_{\text{tot}}$	—	1.0	Variance explained

Table 16 — Evaluation metrics reference.